Advanced Aggregation in PostgreSQL: Statistical Functions & Hypothetical Aggregates

---

Table of Contents

Learning Objectives

By the end of this module, you will be able to:

- Compute percentiles, medians, and statistical measures in SQL
- Use ordered-set and hypothetical-set aggregate functions
- Apply STRING_AGG and ARRAY_AGG for data pivoting
- Aggregate into JSON for API-ready results
- Answer advanced aggregation interview questions

Theory: Beyond Basic Aggregation

PostgreSQL supports a rich set of aggregate functions beyond COUNT/SUM/AVG:

```
Aggregate categories:
─────────────────────────────────────────────────────────────────

Standard aggregates:   COUNT, SUM, AVG, MIN, MAX, STDDEV, VARIANCE
Ordered-set:           PERCENTILE_CONT, PERCENTILE_DISC, MODE
Hypothetical-set:      RANK, DENSE_RANK, PERCENT_RANK, CUME_DIST
Collection aggregates: STRING_AGG, ARRAY_AGG, JSON_AGG, JSONB_AGG
Boolean aggregates:    BOOL_AND, BOOL_OR
Statistical:           CORR, COVAR_POP, REGR_*, STDDEV, VARIANCE
```

Statistical Aggregate Functions

```sql
-- Example 1: Standard deviation and variance
SELECT
    department,
    COUNT(*)                              AS n,
    ROUND(AVG(salary), 2)                 AS mean,
    ROUND(STDDEV(salary), 2)              AS std_dev,
    ROUND(STDDEV_POP(salary), 2)          AS std_dev_pop,  -- population (N)
    ROUND(VARIANCE(salary), 2)            AS variance,
    ROUND(VAR_POP(salary), 2)             AS var_pop,
    MIN(salary)                           AS min_sal,
    MAX(salary)                           AS max_sal,
    ROUND(MAX(salary) - MIN(salary), 2)   AS salary_range
FROM emp_salary
GROUP BY department
ORDER BY std_dev DESC;

-- Example 2: Coefficient of variation (CV = stddev/mean)
SELECT
    department,
    ROUND(AVG(salary), 2) AS mean_salary,
    ROUND(STDDEV(salary), 2) AS std_dev,
    ROUND(100.0 * STDDEV(salary) / NULLIF(AVG(salary), 0), 2) AS cv_pct
FROM emp_salary
GROUP BY department
ORDER BY cv_pct DESC;

-- Example 3: Z-scores (standard scores)
SELECT
    emp_name,
    department,
    salary,
    ROUND(
        (salary - AVG(salary) OVER (PARTITION BY department)) /
        NULLIF(STDDEV(salary) OVER (PARTITION BY department), 0),
        2
    ) AS z_score
FROM emp_salary
ORDER BY department, z_score DESC;
```

PERCENTILE_CONT and PERCENTILE_DISC

These are ordered-set aggregate functions that compute percentiles.

- `PERCENTILE_CONT` — interpolates between values (continuous)
- `PERCENTILE_DISC` — returns an actual data value (discrete)

Syntax

```
PERCENTILE_CONT(fraction) WITHIN GROUP (ORDER BY expression)
PERCENTILE_DISC(fraction) WITHIN GROUP (ORDER BY expression)
```

Examples

```
-- Example 4: Median salary (50th percentile)
SELECT
    department,
    ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary), 2) AS median_salary,
    ROUND(PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY salary), 2) AS median_disc
FROM emp_salary
GROUP BY department
ORDER BY median_salary DESC;

-- Example 5: Multiple percentiles — quartiles
SELECT
    department,
    COUNT(*)                                                        AS n,
    MIN(salary)                                                     AS p0,
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary), 0)  AS p25,
    ROUND(PERCENTILE_CONT(0.50) WITHIN GROUP (ORDER BY salary), 0)  AS p50_median,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary), 0)  AS p75,
    MAX(salary)                                                     AS p100,
    -- IQR (Interquartile Range)
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary)
        - PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary), 0)  AS iqr
FROM emp_salary
GROUP BY department
ORDER BY department;

-- Example 6: Revenue percentiles for anomaly detection
SELECT
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue), 2) AS q1,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue), 2) AS q3,
    -- IQR-based outlier bounds:
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue) -
          1.5 * (PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue) -
                 PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue)), 2) AS
lower_fence,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue) +
          1.5 * (PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY revenue) -
                 PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY revenue)), 2) AS
upper_fence
FROM monthly_revenue;

-- Example 7: PERCENTILE_DISC vs PERCENTILE_CONT difference
-- PERCENTILE_CONT(0.5) interpolates if median falls between two values
-- PERCENTILE_DISC(0.5) returns the lower of the two middle values
SELECT
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY x) AS cont_median,
```

```
    PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY x) AS disc_median
FROM (VALUES (1), (3), (5), (7)) AS t(x);
-- cont = 4.0 (interpolated), disc = 3 (actual value)
```

MODE

MODE returns the most frequently occurring value.

```
-- Example 8: Most common salary in each department
SELECT
    department,
    MODE() WITHIN GROUP (ORDER BY salary) AS mode_salary
FROM emp_salary
GROUP BY department;

-- Example 9: Most common region in sales
SELECT
    MODE() WITHIN GROUP (ORDER BY region) AS most_common_region,
    MODE() WITHIN GROUP (ORDER BY product) AS most_common_product
FROM sales;
```

Ordered-Set Aggregates

These aggregates require an ordering of input rows. They use WITHIN GROUP.

```
-- Example 10: All ordered-set functions
SELECT
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)   AS median,
    PERCENTILE_DISC(0.5) WITHIN GROUP (ORDER BY salary)   AS median_disc,
    MODE() WITHIN GROUP (ORDER BY department)             AS modal_dept,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary)   AS p90_salary
FROM emp_salary;
```

Hypothetical-Set Aggregates

These answer "what would the rank of a hypothetical value be?"

```
-- Example 11: What rank would salary=100000 get?
SELECT
    department,
    RANK(100000)        WITHIN GROUP (ORDER BY salary DESC) AS hyp_rank,
    DENSE_RANK(100000)  WITHIN GROUP (ORDER BY salary DESC) AS hyp_dense_rank,
    PERCENT_RANK(100000) WITHIN GROUP (ORDER BY salary)     AS hyp_pct_rank,
    CUME_DIST(100000)   WITHIN GROUP (ORDER BY salary)      AS hyp_cume_dist
FROM emp_salary
GROUP BY department
ORDER BY department;
```

```sql
-- Example 12: Where would a $90,000 revenue month rank?
SELECT
    department,
    RANK(90000)       WITHIN GROUP (ORDER BY revenue DESC) AS hypothetical_rank,
    COUNT(*)                                               AS actual_months
FROM monthly_revenue
GROUP BY department
ORDER BY department;
```

Statistical Correlation Functions

```sql
-- Example 13: Correlation between salary and tenure
SELECT
    department,
    ROUND(CORR(salary,
        EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date))::NUMERIC, 4
    ) AS salary_tenure_corr
FROM emp_salary
GROUP BY department;

-- Example 14: Regression analysis
SELECT
    ROUND(REGR_SLOPE(salary,
        EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date))::NUMERIC, 2
    )               AS slope_salary_per_year,
    ROUND(REGR_INTERCEPT(salary,
        EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date))::NUMERIC, 2
    )               AS intercept,
    ROUND(REGR_R2(salary,
        EXTRACT(YEAR FROM CURRENT_DATE) - EXTRACT(YEAR FROM hire_date))::NUMERIC, 4
    )               AS r_squared,
    REGR_COUNT(salary, EXTRACT(YEAR FROM hire_date)) AS n
FROM emp_salary
WHERE manager_id IS NOT NULL;
```

STRING_AGG and ARRAY_AGG

```sql
-- Example 15: STRING_AGG — concatenate values
SELECT
    department,
    STRING_AGG(emp_name, ', ' ORDER BY emp_name) AS employees,
    STRING_AGG(emp_name::TEXT || ' ($' || salary::TEXT || ')',
               ' | ' ORDER BY salary DESC) AS salary_list
FROM emp_salary
GROUP BY department
ORDER BY department;

-- Example 16: ARRAY_AGG — collect into array
```

```
SELECT
    department,
    ARRAY_AGG(emp_name ORDER BY salary DESC) AS employees_by_salary,
    ARRAY_AGG(DISTINCT title)                AS unique_titles,
    ARRAY_LENGTH(ARRAY_AGG(emp_id), 1)       AS headcount
FROM emp_salary
GROUP BY department
ORDER BY department;

-- Example 17: Unnest — reverse of ARRAY_AGG
SELECT department, UNNEST(ARRAY_AGG(emp_name)) AS employee
FROM emp_salary
GROUP BY department
ORDER BY department;

-- Example 18: Using array containment for filtering
SELECT department, emp_list
FROM (
    SELECT department, ARRAY_AGG(emp_name) AS emp_list
    FROM emp_salary GROUP BY department
) AS dept_arrays
WHERE emp_list @> ARRAY['Alice CEO'];  -- departments containing Alice
```

JSON Aggregation

```
-- Example 19: JSON_AGG — array of row objects
SELECT
    department,
    JSON_AGG(
        JSON_BUILD_OBJECT(
            'name', emp_name,
            'salary', salary,
            'title', title
        ) ORDER BY salary DESC
    ) AS employees_json
FROM emp_salary
GROUP BY department;

-- Example 20: JSONB_OBJECT_AGG — key-value object
SELECT
    department,
    JSONB_OBJECT_AGG(emp_name, salary ORDER BY emp_name) AS salary_map
FROM emp_salary
GROUP BY department;

-- Example 21: JSON aggregation for API-ready responses
SELECT
    JSON_BUILD_OBJECT(
        'department', department,
        'headcount', COUNT(*),
```

```
        'avg_salary', ROUND(AVG(salary), 2),
        'employees', JSON_AGG(JSON_BUILD_OBJECT('name', emp_name, 'salary', salary)
                              ORDER BY salary DESC)
    ) AS dept_report
FROM emp_salary
GROUP BY department;
```

Window Function Statistical Aggregates

```
-- Example 22: Running statistics
SELECT
    department,
    month_date,
    revenue,
    ROUND(AVG(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS running_avg,
    ROUND(STDDEV(revenue) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ), 2) AS running_stddev,
    COUNT(*) OVER (
        PARTITION BY department
        ORDER BY month_date
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS n
FROM monthly_revenue
ORDER BY department, month_date;
```

ASCII Visual Diagrams

Percentile Visualization

```
Salary distribution for Engineering: [75000, 87000, 88000, 95000, 105000, 120000,
150000]

P0  = 75000    (minimum)
P25 = 87000    (1st quartile)
P50 = 95000    (median)  ← PERCENTILE_CONT(0.5)
P75 = 120000   (3rd quartile)
P90 = 147000   (interpolated)
P100 = 150000  (maximum)

IQR = P75 - P25 = 33000
Lower fence = P25 - 1.5*IQR = 87000 - 49500 = 37500
```

```
Upper fence = P75 + 1.5*IQR = 120000 + 49500 = 169500
No outliers in this dataset
```

CORR Values

```
CORR(x, y) ranges from -1.0 to +1.0:

-1.0 ──────────── 0 ──────────── +1.0
 Perfect          No          Perfect
 negative     correlation     positive
 correlation                 correlation

|CORR| > 0.8: Strong correlation
|CORR| 0.5-0.8: Moderate
|CORR| < 0.5: Weak
```

Common Mistakes

Mistake 1: Confusing STDDEV and STDDEV_POP

```
-- STDDEV (= STDDEV_SAMP): uses N-1 denominator (sample)
-- STDDEV_POP: uses N denominator (population)
-- Use STDDEV_POP when computing for the complete dataset
-- Use STDDEV (sample) when estimating from a sample
```

Mistake 2: PERCENTILE_CONT returning NULL for empty set

```
-- PERCENTILE_CONT returns NULL when the group is empty
-- Protect with COALESCE:
COALESCE(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary), 0)
```

Mistake 3: WITHIN GROUP ORDER BY direction matters

```
-- PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary ASC)
-- = 90th percentile (top 10% threshold)

-- PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY salary DESC)
-- = 10th percentile (different!)
-- Order direction changes the meaning of the fraction!
```

Best Practices

1. Use PERCENTILE_CONT for median and percentile calculations — it is more accurate than custom ranking.
2. Use STDDEV_POP when you have the entire population; STDDEV for samples.
3. Use JSON_AGG for API-ready aggregated results (eliminates application-layer assembly).

4. Use STRING_AGG ORDER BY for deterministic output.
5. Prefer ARRAY_AGG over STRING_AGG when downstream code needs to parse the list.

Performance Considerations

```
-- PERCENTILE_CONT requires sorting the input — expensive for large tables
-- Pre-sort using an index:
CREATE INDEX idx_emp_salary_dept ON emp_salary(department, salary);

-- Materialized views for expensive statistical aggregations:
CREATE MATERIALIZED VIEW dept_statistics AS
SELECT
    department,
    COUNT(*)                                                    AS n,
    ROUND(AVG(salary), 2)                                       AS mean,
    ROUND(STDDEV(salary), 2)                                    AS std_dev,
    ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary), 2) AS median,
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary), 2) AS p25,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary), 2) AS p75
FROM emp_salary
GROUP BY department;

REFRESH MATERIALIZED VIEW dept_statistics;
```

Interview Questions & Answers

Q1: How do you compute the median in PostgreSQL?

A: Use `PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY column)`. This is an ordered-set aggregate that interpolates the 50th percentile. PERCENTILE_DISC(0.5) returns an actual data value (the lower middle value for even counts).

Q2: What is the difference between PERCENTILE_CONT and PERCENTILE_DISC?

A: PERCENTILE_CONT interpolates between data points for fractional positions, returning a value that may not exist in the data. PERCENTILE_DISC always returns an actual value from the dataset — the smallest value whose rank is >= the specified fraction.

Q3: What are hypothetical-set aggregates?

A: They answer "what rank/percentile would this hypothetical value have in the current group?" Syntax: `RANK(value) WITHIN GROUP (ORDER BY column)`. Useful for what-if analysis without inserting data.

Q4: What is the difference between STDDEV and STDDEV_POP?

A: STDDEV (= STDDEV_SAMP) divides by N-1 (Bessel's correction), used for sample data. STDDEV_POP divides by N, used when you have the entire population. For most business queries, STDDEV is appropriate.

Q5: How does STRING_AGG work?

A: STRING_AGG(expression, delimiter) concatenates non-NULL values in each group using the specified delimiter. An ORDER BY inside the aggregate controls the concatenation order. NULL values are ignored.

Q6: How is JSON_AGG useful in PostgreSQL?

A: JSON_AGG collects rows into a JSON array, allowing you to return hierarchical data in a single query. Combined with JSON_BUILD_OBJECT, you can produce API-ready responses that eliminate N+1 queries in application code.

Q7: What is CORR() used for?

A: CORR(y, x) computes the Pearson correlation coefficient between two columns, measuring the strength and direction of their linear relationship. Returns a value between -1 (perfect negative correlation) and +1 (perfect positive correlation).

Q8: What is the WITHIN GROUP clause?

A: WITHIN GROUP is used with ordered-set and hypothetical-set aggregates to specify the ORDER BY for the aggregate computation. It is distinct from the query's ORDER BY — it controls the ordering within each aggregate group.

Hands-on Exercises

Exercise 1: Salary Statistics Dashboard

For each department, compute: count, mean, median, IQR, and coefficient of variation.

```
-- Solution:
SELECT
    department,
    COUNT(*)                                                          AS n,
    ROUND(AVG(salary), 0)                                             AS mean,
    ROUND(PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary), 0)     AS median,
    ROUND(PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary), 0)    AS q1,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary), 0)    AS q3,
    ROUND(PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) -
          PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary), 0)    AS iqr,
    ROUND(100.0 * STDDEV(salary) / NULLIF(AVG(salary), 0), 2)         AS cv_pct
FROM emp_salary
GROUP BY department
ORDER BY department;
```

Exercise 2: Employee JSON Report

Create a JSON report per department with all employee details.

```
-- Solution:
SELECT
    department,
    JSON_BUILD_OBJECT(
        'headcount', COUNT(*),
        'payroll', SUM(salary),
        'avg_salary', ROUND(AVG(salary), 2),
        'employees', JSON_AGG(
            JSON_BUILD_OBJECT('name', emp_name, 'title', title, 'salary', salary)
```

```
            ORDER BY salary DESC
        )
    ) AS dept_summary
FROM emp_salary
GROUP BY department
ORDER BY department;
```

Exercise 3: Outlier Detection

Find all employees whose salary is more than 1.5 IQR above Q3 or below Q1 (IQR outlier method).

```
-- Solution:
WITH stats AS (
    SELECT
        department,
        PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary) AS q1,
        PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) AS q3,
        (PERCENTILE_CONT(0.75) WITHIN GROUP (ORDER BY salary) -
         PERCENTILE_CONT(0.25) WITHIN GROUP (ORDER BY salary)) AS iqr
    FROM emp_salary GROUP BY department
)
SELECT e.emp_name, e.department, e.salary,
       ROUND(s.q1, 0) AS q1, ROUND(s.q3, 0) AS q3,
       CASE WHEN e.salary < s.q1 - 1.5 * s.iqr THEN 'Low Outlier'
            WHEN e.salary > s.q3 + 1.5 * s.iqr THEN 'High Outlier'
            ELSE 'Normal'
       END AS outlier_status
FROM emp_salary e
JOIN stats s ON e.department = s.department
ORDER BY e.department, e.salary;
```

Advanced Notes

Ordered-Set Aggregate with FILTER

```
-- Conditional percentile: median salary for employees hired after 2020
SELECT
    department,
    PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY salary)
        FILTER (WHERE hire_date > '2020-01-01') AS recent_hire_median
FROM emp_salary
GROUP BY department;
```

Custom Aggregates

PostgreSQL allows defining custom aggregates with CREATE AGGREGATE, enabling domain-specific computations not built into the engine.

Cross-references

- **02_aggregations.md** (03_Intermediate_SQL) — Basic aggregate functions
- **07_grouping_sets_rollup_cube.md** (03_Intermediate_SQL) — Multi-level aggregation
- **01_window_functions_intro.md** — Statistical functions as window functions
- **08_interview_window_tasks.md** — Interview problems using statistical functions